Thruput Manifesto

GEOFFREY RAMSEYER, ADAM CHAMBERS, CHRIS FROST, DANIEL GIFFIN, JACK HUMPHRIES, ADRIAN LANZAFAME, GIULIANO LOSA, PETAR MAYMOUNKOV, MARK MAYO, DAVID MAZIÈRES, VINITA PAWAR, and RYAN STUTSMAN

Stellar Development Foundation

`https://stellar.org/thruput`

Abstract: Thruput is a blockchain whose global transaction bandwidth scales linearly with hardware resources. This relief from computational scarcity permits the network to operate sustainably without imposing any fees on users' smart-contract gas usage, radically expanding the network's practical utility. Thruput achieves this by allowing each block's transactions to be processed in parallel. Logically, the transactions execute simultaneously, and many transactions may update the same persistent storage location with commutative operations. Block proposers preserve datatype invariants such as integer non-negativity for token balances by dynamically excluding transactions that prove incompatible, allowing contracts to manipulate shared state safely without explicit concurrency controls. Because committed state evolves only at block boundaries, a node's many code-execution units can cache storage values without coherency concerns, permitting a highly distributed design for the persistence service as well. Experiments show that a distributed validator can achieve nearly linear scaling of transaction throughput over cores (measured here up to 64) and machines (up to 8).

1 Introduction

Digital money has no excuse for being worse than physical cash: it should be free to use, open to anyone, and entirely under its owner's control. The systems we use to pay online—cards, bank transfers, and payment apps—meet none of these conditions. Gatekeepers control access, owners surrender custody, and fees are charged for what is, at its core, sending a message.

This is an absurd state of affairs.

Blockchains promised to fix that, and in some ways they have. Decentralization and replication mean that a blockchain keeps running even when nodes suffer hardware failures or regional networks go down. Smart contracts let anyone publish rules for how their assets move and know the whole network will enforce them. Taking part requires only a network connection and a device that can compute a signature. These are real achievements.

And yet, today's blockchains are still slow and expensive to use. Blockchains are forced to charge increasingly high fees as demand to use them increases, because they cannot easily scale up the volume of transactions they can handle. This limitation is not a question of just improving the engineering, but is rather a fundamental limitation of their architectures.

Our mission with Project Thruput is to eliminate every practical scaling limit so that we can make smart contract execution free.

1.1 The Curse of Ordering

The root of the problem is that blockchains pretend to execute transactions sequentially. This is a natural design choice, and makes it easy to ensure that smart contract execution produces the same results on every node. Developers are used to writing sequential, "single-threaded" code.

Yet there is only so much sequential computation that can be done within one unit of time, because we cannot buy arbitrarily fast CPU cores. The common choice is to subdivide a unit of time into more, shorter slices, meaning more transactions but less compute per transaction. That limits the complexity of smart contracts and pushes developers into arcane tricks to hyperoptimize smart contract computation [3, 35, 80]. Adding CPU cores can accelerate workloads where transactions

rarely modify shared state, but when one transaction depends on the result of another, the second must wait for the first. Yet shared state is central to the financial markets that blockchains enable. Blockchains regularly see many users transacting on the same market, or many buyers paying one merchant, or moving assets through the same DeFi protocol.

Additional CPUs fundamentally cannot provide more than a bounded speedup on these ordered workloads. Boosting throughput therefore requires increasingly sophisticated engineering projects in pursuit of ever-diminishing marginal returns. Imposing artificial divisions on state, as in sharded blockchains and Layer-2 systems, merely hides the problem of managing shared resources (cross-shard transactions become even more expensive) and makes user experience meaningfully worse. No user should need to think about what shard their money lies on.

As a result, what a sequential blockchain delivers is a system that is expensive to use, can barely realize the full potential of smart contract customization, and cannot support concurrent usage by everyone around the world.

1.2 Project Thruput

Thruput demonstrates that blockchains need not, in fact, totally order all transactions. In Thruput, blocks remain ordered relative to each other, but all transactions within one block semantically execute **at the same logical instant**. Instead of optimistically parallelizing sequential work, Thruput's implementation executes transactions in a totally unpredictable, interleaved, and unspecified physical order.

And yet, at the end of each block, all Thruput nodes agree on the state of the blockchain and what each transaction did. Thruput achieves deterministic execution not by relying on a shared ordering, but by changing how contracts modify persistent state. Smart contracts write through APIs that make concurrent transactions *commute*: their updates produce the same aggregate result regardless of the order in which they are applied. The key insight is that these APIs permit relaxed ordering while still making it practical to implement nearly all existing financial primitives expected of a blockchain. When two addresses transfer tokens to the same recipient, those operations should commute as long as the sum of the token transfers and the recipient's current balance is not *negative*. Blocks are constructed carefully to preserve key invariants, like non-negative token balances. Contracts encapsulate these APIs within familiar interfaces. For example, a contract transfers tokens by calling the token contract's "Token.transfer()" method. Contracts that need sequential operations can serialize their own requests without imposing that order on the rest of Thruput (§4.3).

This design enables Thruput to scale up in the simplest way possible: adding additional CPUs to process transactions [27]. Each Thruput node is implemented as a distributed system (§3), with transaction execution across several machines. Our implementation scales transaction throughput **linearly** with hundreds of CPU cores. Doubling the number of cores nearly doubles the throughput. This design fundamentally changes the tradeoff between blockchain throughput and capacity planning for users of the chain. We do not claim any level of maximum throughput; instead, our design means one can calculate how many CPUs are needed for whatever level of throughput is desired.

Through all of this, Thruput looks like a typical blockchain from the outside. Each node in the system maintains a copy of the chain. There is no sharding of state or artificial subdivision between users, and no extra security requirements beyond the standard honest-supermajority assumptions of consensus protocols. Thruput imposes no additional requirements on end users.

Thruput's scalability lets it realize the vision of a global, open, free, and programmable financial infrastructure. Because it is not bound to any fixed transaction rate, Thruput scales compute supply arbitrarily in response to demand and need not charge a transaction fee to limit demand. By

eliminating on-chain compute scarcity, Thruput enables new classes of smart contracts that are impractical to run on today's chains. Extreme events in one part of the world or bursts of activity in a memecoin do not crowd out normal payment activity (§12.2). Services may charge fees for value they provide on top of the digital money system, but the smart contracts themselves can be free to operate (§12).

Project Thruput is under active development. It extends prior work [77], realizing a vastly more capable system with a fully distributed architecture. We describe here the technical innovations behind Thruput, outline the current state of development, and survey our technical roadmap.

2 Thruput Smart Contract Overview

Like any blockchain, smart contract execution on Thruput must be entirely deterministic to be replicable across every node in the system. Each Thruput transaction invokes a smart contract, with some input calldata. Smart contracts in Thruput are implemented as RISC-V programs (`riscv32-im`; §8.1) that run inside an optimized virtual machine. Persistent storage is a key-value store with 64-byte keys and values with structured types (§4); contracts access this store through structured system calls. Each contract is deployed at a 32-byte "ContractAddress" and owns a 32-byte keyspace within the key-value store (so every item is indexed by 'ContractAddress || StorageKey'). One contract can synchronously invoke another contract within the same transaction. Contracts emit events to signal actions to external systems.

What makes Thruput unique lies in how transactions interact with that key-value store. In a traditional, sequential blockchain, values written in one transaction can be read in the next transaction. Maintaining coherence about this ordering is the core engineering challenge when optimistically executing transactions in parallel (Block-STM [39]). Thruput does not need a complex concurrency management protocol. Instead, it changes how transactions access shared state.

Snapshot Isolation. All transactions in a Thruput block read from the same snapshot of the database, as it existed at the end of the previous block. A transaction's side effects cannot be observed by any other transaction in the same block; they are observed only by transactions in future blocks.[1] These semantics are similar to Snapshot Isolation in traditional database designs.

Write Contention. If every transaction wrote to distinct keys, Thruput would have no problem reconciling concurrent writes to produce the database snapshot for the next block. The interesting use cases for blockchains, however, are exactly those where contention on shared data is frequent and inherent. Markets, and especially automated market-makers, sequence trade orders and offer the best price to whichever trade is executed first. NFT drops often allocate a limited number of objects to the first buyers. Even simple payments experience contention on high-activity accounts, such as flows in and out of exchanges and banks, or payments to the tax authorities on tax day.

To send a payment in a standard blockchain environment, a token contract reads the sender's balance, subtracts from it, aborts if that balance is negative, and writes an updated balance. It then reads the receiver's balance, adds to it, and writes back that update. The blockchain node executing the contract sees an opaque sequence of bytecode operations, along with two reads and two writes to the key-value store; it has no knowledge of the algebraic relationship between the operations.

Thruput offers new primitives that facilitate the resolution of concurrent writes. Instead of simply writing an updated balance into an opaque storage slot, the token contract asks the runtime to "add X" to the integer-typed value in the slot representing the receiver's balance, and "subtract X" from the value in the sender's slot. (Previous integer-valued writes to each location imply to the runtime what types they have.) When multiple transactions modify the same balance (if,

[1] With limited exceptions; see §5.

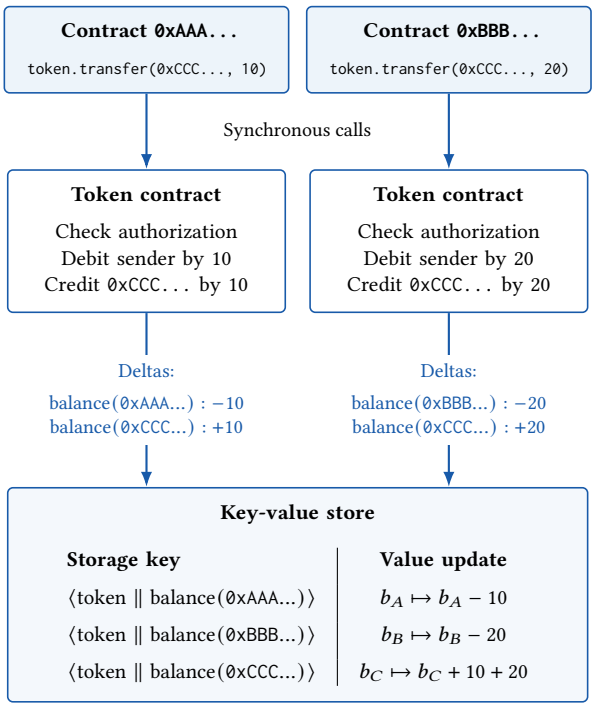


Fig. 1. **Two transactions in the same block.** The token contract wraps integer update calls in an ordinary `transfer` API, which other contracts can synchronously call. Two transactions (left and right) both modify the balance of address `0xCCC...`, and the Thruput runtime mediates the concurrent access.

for example, one user sends or receives many payments at once), the runtime simply sums all the additions and subtractions to compute the value for the end-of-block snapshot. Because the operations are commutative, every node will agree on that snapshot. This commutativity allows Thruput to achieve deterministic outcomes without suffering the synchronization overhead of keeping CPUs executing in an agreed order.

Figure 1 illustrates how a token contract can encapsulate these primitives behind a familiar `transfer` API. Other contracts can invoke it with ordinary synchronous cross-contract calls.

Commutativity, however, is insufficient to actually write useful applications on a blockchain.[2] Balances of a token, for example, cannot be allowed to become negative. Thruput therefore enforces constraints on the aggregate behavior of transactions in a block (§2.1).

Each value type (detailed in §4) in Thruput has two associated properties.

Typed Commutative Interface. Contracts read and modify values via a structured API. Each modification (like an "add" to an integer) is associated with a particular type of value. These

[2]Without the invariants described, the Thruput state model would look like an application of Conflict-Free Replicated Data Types (CRDTs) [83]. A CRDT cannot directly enforce a non-negative balance, because a replica subtracting from a balance cannot know whether other, not-yet-applied operations also subtract from it. Escrow-based variants [10] sidestep that limitation by partitioning the balance among a fixed set of replicas ahead of time and having each replica refuse subtractions it cannot locally cover, which is akin to sharding the blockchain across a fixed, finite set of subnetworks. By contrast, Thruput must reconcile only a finite set of transactions in one block, so it can admit subtractions on demand from a single reservation table and defer the rest.

structured modifications allow the Thruput runtime to mediate writes across multiple concurrent transactions.

Value Invariants. The Thruput runtime maintains specific invariants (defined by the value type) on modifications to a value by transactions in the same block. These prevent the kinds of semantic conflicts, like a balance going negative, that cannot be addressed within one transaction in Thruput's parallel state model.

A contract writer can choose to ignore these special datatypes (serializing all stored data into the basic "bytestring" type, §4.1), but this leaves Thruput unable to resolve concurrent writes; it would have to exclude all but one transaction on the contract per block, in the worst case. Using the concurrency-sensitive datatypes, a contract can handle worst-case contention patterns like millions of users simultaneously sending payments to a single address. There is furthermore a general transformation (see §4.3) that allows any contract to preserve strictly sequential semantics while servicing concurrent invocations. A queue consumer processes its requests sequentially on one CPU, while other transactions can run on other executor cores. This ordering is local to that queue; it does not order other transactions that access the same contracts. These workloads still share finite compute, storage, and network capacity, and their writes can contend on shared state.

Similarly, because a contract modifies state via structured system calls, the remainder of the computation can be performed by arbitrary, untrusted bytecode. Thruput requires no static analysis or approval process to guarantee commutativity. Like any other blockchain, smart contracts on Thruput are uploaded by untrusted users and may be actively malicious. A bug in one contract cannot corrupt the rest of the chain.

2.1 Proposing and Validating a Block

Transaction execution proceeds in two steps. First, a Thruput node runs the specified smart contract invocation, executing RISC-V instructions, reading from the database, and accumulating output writes as necessary.

Then, Thruput performs a two-phase commit process to apply the transaction's writes to the database. For each key, the transaction first `reserve()`s the right to apply the modification to the database. Thruput uses a fine-grained locking scheme, depending on the type of the value. For example, for a token balance (non-negative integer, §4.2) with snapshot value of 100, Thruput allows up to 100 units worth of subtractions to be `reserve()`d at the same time. If all reservations can be acquired successfully, the transaction then `commit()`s each of its writes.

Thruput uses a rotating leader to propose each new block. When proposing a block, a Thruput node executes any pending transactions in its mempool, and greedily packs in any that succeed at all their reservations. Those that fail are excluded, but can be retried in future blocks.[3] Validating a proposal means executing all transactions and verifying that no reservations failed. Block validity is a property of the set of transactions (and some proposer metadata; see §5), not the order in which they touch the database. Consensus rejects proposals that fail validation. The reserve-commit process guarantees that no honest proposer can accidentally produce an invalid block, and guarantees that an honest block proposal satisfies the type-specific invariants for all values in persistent storage. There is no extra combinatorial problem to solve to ensure that the type-specific invariants are all satisfied, and both block proposal and validation are entirely parallelizable.

[3]Which conflicting transactions to exclude is up to the proposer (and might depend on nondeterministic inter-CPU scheduling), but block execution is deterministic once a node stamps a block proposal.

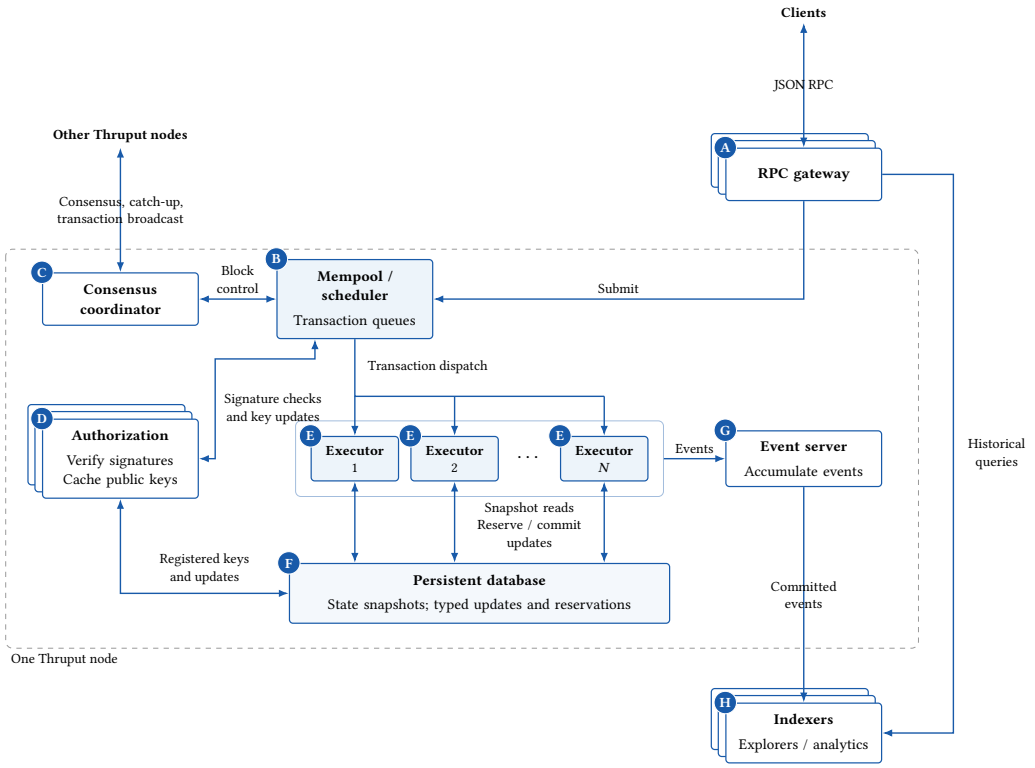


Fig. 2. Data flow within one Thruput logical node.

3 System Architecture

Thruput is built from the ground up to be a distributed system, where the key components can independently scale. Figure 2 outlines the architecture of one node.

External RPC Gateways (A) mediate access from external clients, who might send transactions via a JSON API or subscribe to a websocket feed of events. Transactions are forwarded to a mempool process (B), which buffers pending transactions locally. A consensus coordinator (C) manages communication with other Thruput nodes, handles consensus messages, and answers requests for past blocks from nodes catching up to the chain. It also manages a multicast broadcast subsystem to forward new transactions from the mempool to other Thruput nodes.

Adjacent to the mempool is an authorization service (D), which is responsible for the (expensive) cryptographic signature verification on each transaction. Every address can configure which public keys authorize transactions that start with an invocation of the contract at that address (§7). The authorization service must load these registered keys from the persistent database (F). The authorization service is stateless, but maintains a cache of recently used public keys. The mempool stores transactions that pass authorization on a queue of transactions that are ready to execute. When a block of transactions commits, some addresses might have changed their registered public keys, so the database notifies the authorization service and the mempool of all addresses that must have their cached keys invalidated and authorization re-checked.

When a Thruput node is called upon to propose a new block, the mempool dispatches transactions from its run-queue to transaction executors (E). These stateless services execute transactions as

they arrive (§8), reading and writing to the database as necessary (§9). Thruput's snapshot-read model allows the executors to trivially cache values read from the database. Furthermore, when a transaction reads from the database, that access reads data not in local memory but on a remote machine, incurring nontrivial latency. But during that remote access, the executor's CPU can simply switch to executing another ready transaction (§8.2), so CPU time is never wasted. In a standard sequential chain, by contrast, that remote access could stall every other transaction in the block.

The mempool prioritizes transactions to execute based on the number of native tokens held at each address (§12.2). When a block commits, the database notifies the mempool of native token transfers.

Transactions may emit events, which are accumulated on an event server (G). When a block commits, these events are forwarded to downstream indexers (H), which might include analytics services.

4 Concurrent Data Types

We outline here the data types and APIs that Thruput smart contracts use to read and modify persistent state. This is not a complete API reference, and we may add additional types in the future. There are also system calls for emitting events, verifying a signature, and querying which address called the current method ('msg.sender').

We design the Thruput semantics to resolve as many conflicts as logically possible by default. However, any contract that needs exclusive access to a particular value can add an exclusive lock flag to any storage write.

4.1 Bytestrings

The minimal, default way for a contract to write data is as an opaque bytestring. This is how most blockchains operate. Yet an opaque bytestring gives the Thruput runtime no information about how to combine concurrent writes to the same key. Without any information, Thruput cannot do anything but reject transactions that write different values to the same key in a block. By default, Thruput admits pairs of transactions that write the same string to a key. This behavior can be overridden by the exclusive lock flag.

A contract's stored configuration data, such as an administrator address, fits well in this model and has no meaningful notion of commutativity to express to Thruput. One might use the exclusive lock flag when initializing a new contract, to ensure that stateful operations associated with the initialization (like transferring funds) are only performed once. In other settings, such as computing the closing time of an auction and updating a state variable to mark an auction as closed, multiple transactions can make that write and proceed concurrently.

4.2 Nonnegative Integers

Financial applications need some notion of a balance that cannot be negative. For these cases, Thruput provides a "non-negative integer." The natural API for a token balance would be to add to and subtract from it; Thruput gives a generalized version of that API, which we call an `nnint_set_add(base, addend)` operation. Here `base` and `addend` are any two *signed* 128-bit integers. Logically, this operation sets the value of the integer to `base`, and then adds `addend` to that base value. The corresponding `nnint_get` API returns the (`base`, `addend`) pair most recently written to that key in the current transaction, or the snapshot value (and a `0` `addend`) for values not yet modified by the transaction. This API can implement both token balances and more general linear constraints on financial operations, like collateralized borrowing limits and spending velocity constraints.

Token Implementation

```
fn balance_of(owner) -> u128 {
    key = balance_key(owner);
    (base, delta) = nnint_get(key);
    return (base + delta) as u128;
}

fn transfer(to, amount) {
    require(amount > 0);
    sender = msg.sender;
    _adjust(balance_key(sender), -amount);
    _adjust(balance_key(to), amount);
    emit(Transfer { sender, to, amount });
}

fn _adjust(key, change) {
    (base, delta) = nnint_get(key);
    delta += change;
    require(base + delta >= 0);
    nnint_set_add(key, base, delta);
}
```

Pseudocode; a partial ERC20 implementation

Transfer Velocity Limit

```
fn spend(to, amount) {
    require(msg.sender == owner);
    require(amount > 0);
    epoch = block_height() / WINDOW;

    if value_get(EPOCH) != epoch {
        value_set(EPOCH, epoch);
        (base, delta) = (LIMIT, 0);
    } else {
        (base, delta) = nnint_get(BUDGET);
    }

    delta -= amount;
    require(base + delta >= 0);
    nnint_set_add(BUDGET, base, delta);
    token.transfer(to, amount);
}
```

Pseudocode; a fixed-window spending cap. `owner`, `token`, `LIMIT`, and a positive `WINDOW` are configured at deployment. A missing epoch marker triggers the first reset. `value_get` and `value_set` manipulate bytestrings holding encoded values.

Fig. 3. Two uses of concurrent integer updates. Left: an ERC20-style token interface. Right: a wallet enforcing a transfer limit per window. The runtime checks aggregate balance and budget constraints across transactions, while contracts expose ordinary synchronous methods. Pseudocode omits arithmetic overflow checks.

Constraint: Nonnegativity. Two concurrent transactions that produce `nnint_set_add`(B_1, A_1) and `nnint_set_add`(B_2, A_2), respectively, conflict if $B_1 \neq B_2$ (analogously to the bytestring case, §4.1). A collection of operations `nnint_set_add`(B, A_1), . . ., `nnint_set_add`(B, A_n) is invalid (i.e., reservations of these operations cannot all succeed) if $B + \sum_{i=1}^{n} A_i < \min(0, B)$; that is, if the value of the integer becomes negative. In the interest of maximizing flexibility for contracts, we allow $B < 0$ (as contracts can choose to abort before making an `nnint_set_add` system call with such an argument), in which case the constraint means that the value cannot become further negative. Thruput also enforces an analogous constraint against overflowing the maximum value of a 128-bit integer. 128 bits is an arbitrary choice, but we believe it provides a reasonable tradeoff between arithmetic width and implementation complexity.

Reservations track positive and negative adjustments separately against the agreed base value. The implementation is conservative in that a credit in one transaction does not increase the capacity available for another transaction's debit in the same block, so really Thruput enforces that collection of `nnint_set_add`operations satisfies $B + \sum_{i=1}^{n} \min(A_i, 0) < \min(0, B)$. For example, with a base balance of 100, a credit of 100 cannot make room for two concurrent debits of 80, even though the net result would be 40.

4.2.1 Example: Token Contract. A token contract can represent balances (and allowances) as nonnegative integers. A balance adjustment is implemented by reading a user's snapshot balance and then performing an `nnint_set_add()` operation, giving the just-observed balance as the base value.

Borrowing and Repayment

```
fn borrow(amount) {
    require(amount > 0);
    user = msg.sender;
    _change_debt(user, amount);
    token.transfer(user, amount);
}

fn repay(amount) {
    require(amount > 0);
    user = msg.sender;
    _change_debt(user, -amount);
    token.transfer_from(user, self.address,
        amount);
}
```

Shared Borrowing Constraint

```
fn _change_debt(user, change) {
    dkey = debt_key(user);
    mkey = margin_key(user);

    (debt, delta) = nnint_get(dkey);
    delta += change;

    margin = snapshot_limit(user) - debt;

    // Debt cannot be negative
    nnint_set_add(dkey, debt, delta);
    // If margin < 0, debt can only be repaid
    nnint_set_add(mkey, margin, -delta);
}
```

Snapshot helpers ignore this transaction's pending writes; `delta` accumulates its debt changes. The borrowing limit values collateral using the block's fixed prices and discount factors.

Fig. 4. Collateralized borrowing as a concurrent linear constraint. Interest accrual calculations omitted.

Figure 3 sketches pseudocode for part of an ERC20-style [96] interface. Other contracts request transfers via synchronous cross-contract calls to the token contract, without knowing how it is implemented.

4.2.2 Example: Transfer Velocity Limit. The base value argument to `nnint_set_add()` lets the operation support other uses as well. A transfer velocity limit, for example, can be implemented as a non-negative integer whose base value resets at regular intervals.

Figure 3 shows a wallet that permits at most `LIMIT` tokens to be transferred during each window of `WINDOW` blocks. At a window boundary, concurrent calls write the same epoch marker and reset the budget to the same base value; their spending deltas combine. The budget update and token transfer happen within one transaction.

4.2.3 Example: Collateralized Lending. A collateralized borrowing/lending protocol (such as Compound [28], Blend [17], Aave [1]) allows users to post collateral (typically some volatile asset) and borrow another asset against it (typically a stable asset, like a USD-pegged stablecoin). The amount that can be borrowed is a complicated calculation, which might include pricing the user's posted collateral with various protocol-defined discount factors based on, for example, asset volatility. Interest rates similarly float (typically) in response to a variety of factors, such as the amount of an asset posted as collateral relative to the amount of that asset borrowed.

To implement such a protocol on Thruput, a contract would compute updates to interest rates and collateral valuations once per block. Compound already uses a related pattern on a sequential chain: its interest indices accrue only when the block timestamp advances [29]. Thruput can then implement borrowing headroom and total debt as non-negative integers. Figure 4 sketches a lending market's borrow and repayment paths. The contract computes the borrowing limit available to a user, minus its existing debt, and uses that margin as the base value of the `nnint_set_add()` call.

These loans can become undercollateralized, when, for example, the value of the collateral asset crashes. Each protocol has some mechanism for absorbing those losses; that choice is not especially relevant here. What matters for Thruput is that these undercollateralized states are still representable, so that contracts can make forward progress to resolve them.

This example demonstrates why we allow the base value of `nnint_set_add()` to be negative: Thruput enforces only the constraints that cannot be enforced by a transaction itself. A transaction cannot directly observe concurrent operations in other transactions; instead, contracts express value invariants to Thruput so the runtime can enforce them. A transaction *can* observe states written in prior blocks, so Thruput places as few constraints as possible on how contracts operate on past state.

4.3 Retrocausal Queues

Some contracts, such as an automated market maker or an orderbook,[4] require strictly sequential semantics. For these contracts, Thruput provides a multi-producer, single-consumer queue. Queues allow a consumer transaction to enforce a strict ordering on request processing without forcing the whole chain into a single total order.

Constraint: Unique Consumer. A caller of the contract can insert a message into a queue. Insertions do not conflict with each other, and the queue is FIFO-ordered across blocks (and lexicographically sorted within past blocks). One transaction can then consume entries from the queue one by one, and only one transaction is allowed to consume from the queue in a block. This means that every entry that is placed into a queue can be consumed at most once, and entries are never dropped unless the queue is explicitly deleted.

Retrocausality. Thruput's snapshot isolation would ordinarily impose a delay of at least one block between when a producer enqueues an item and the consumer can observe the effects. We eliminate this delay by relaxing snapshot isolation for queues. An item may be dequeued in the same block that it is enqueued, and the order in which concurrently enqueued items are dequeued can depend on nondeterministic parallel scheduling in the block proposer. As a result of this relaxation, block validators may need to dequeue an item before executing the corresponding concurrent enqueue operation. Queues are "retrocausal" because Thruput only enforces the invariant that any item dequeued must have been enqueued by the end of the block.

Since the order of dequeuing items enqueued in the same block is nondeterministic, the proposer must communicate that order to validators. Thruput does so by including *proposer data* in blocks. The details are in §5.2.

4.3.1 Example: AMM Contract. An automated market-maker such as Uniswap [2] maintains reserves of two assets and allows users to exchange either asset for the other, at an exchange rate that adjusts based on the amounts of assets in the AMM. The price calculation formula is different in each implementation, but generally the price changes after every trade. Thus trades do not commute with each other (each must observe the price change resulting from the previous trade), and the AMM needs to execute them in a strictly sequential order.

The contract's trading code would achieve this sequential behavior naturally, because writing the next price (*e.g.*, as the base value of a `nnint_set_add` operation) would not commute with other transactions. But this would limit the application to servicing only one trade per block. To handle an acceptable amount of volume, it would need to service many (sequential) trades in each block. Thruput can support this pattern by wrapping calls to the AMM in a queue. Callers submit

[4] A decentralized exchange can be implemented in Thruput's concurrency model, but this requires substantial changes to the trading mechanism [79], which touches on long-running open questions in the economics literature (e.g., [19, 20, 46, 47, 54]).

Submit a Swap

```
fn swap_x_for_y(amount, min_out) {
    require(amount > 0 && min_out >= 0);
    user = msg.sender;

    request = (user, amount, min_out);
    queue_insert(REQUESTS, request);
}
```

Process Queued Swaps

```
fn crank() {
    amm_address = get_current_address();
    (x, y) = value_get(RESERVES);

    for _ in 0..MAX_BATCH {
        request = queue_consume(REQUESTS)
            else break;
        (user, amount, min_out) = request;

        token_x.transfer_from(user,
            amm_address, amount);

        // Uniswap v2's 0.3% swap fee
        net = 997 * amount;
        out = (net * y) / (1000*x + net);
        if out == 0 || out < min_out {
            // Refund
            token_x.transfer(user, amount);
            continue;
        }

        (x, y) = (x + amount, y - out);
        token_y.transfer(user, out);
    }

    value_set(RESERVES, (x, y));
}
```

Fig. 5. A simple Uniswap v2-style swap operation. Request failure handling is omitted; see §5.1.

trade requests to the queue, and then a separate transaction "turns the crank" to pop elements from the queue and apply them to the AMM.[5] Figure 5 gives a simplified example using Uniswap v2's [2] pricing rule.

There are several limitations to this design. The first is that unlike the rest of Thruput, there is a limit to the number of transactions that a single AMM can handle in a block. This means that an AMM may wish to charge a transaction fee (using, e.g., an EIP-1559 style dynamically adjusted fee) to limit the number of enqueued swaps to a rate that can be sequentially processed. In other blockchains, this fee mechanism is handled by the consensus protocol; here, it must be implemented (but the fee revenue can then be paid directly to the liquidity providers of the AMM). Second, calls into an AMM cannot be completed synchronously by other contracts, which limits what can be done within a single transaction.

And finally, synchronous calls that the AMM makes (such as token transfers) might still conflict with other, concurrent transactions. An AMM could handle this by transferring the token sold when the request is enqueued; then, assuming the token contract actually satisfies the token contract

[5]This notion of cranking a queue is a well-established pattern, as in, for example, [49, 75].

specification, the work of processing one trade would not depend on whether the trader still has sufficient funds at the time the request is executed.

However, that design requires case-by-case analysis and may be difficult to implement in practice. Thruput also introduces a general-purpose mechanism, commit locking (§5.1), to allow contracts to capture a few limited bits of information about concurrent transactions. With commit locking, the queue crank operation is a general-purpose mechanism for any sequential contract, which requires no specific assumptions or case-by-case analysis.

4.4 Replay Nonce Set

Thruput prevents transaction replay with a (nonce, expiry height) replay tuple on each transaction. Thruput will never commit two transactions from the same contract address with identical replay tuples, but does not require that nonces on an address increase sequentially. Allowing gaps in nonces lets Thruput execute many transactions starting at some address in the same block. Consumed replay tuples are garbage collected when the chain reaches the expiry block height.

We intend to give contracts the ability to manipulate sets of nonces directly. One might, for example, have a separate notion of replay nonce for so-called "intent" mechanisms (such as [64]), but the implementation is not yet ready.

4.5 Roadmap: Other Data Types

One can imagine a number of other potentially useful data types for contracts in the Thruput model. Our early research prototypes [77] combined replay nonces and queues into an "ordered set" primitive. SPEEDEX [79], which operates in an analogous concurrency model, used a notion of a bitmap to prevent transaction replay, and used a specialized ordered set to implement an orderbook.

The first version of Thruput supports the data types described in this section because we believe them to be sufficient to implement a wide variety of real-world applications. We may add additional types in response to developer feedback.

5 Nondeterministic Proposer Data

A naïve queue abstraction in Thruput would pose two problems for contracts that want to serialize request handling. First, as alluded to in §4.3, snapshot isolation would force a one-block delay between when an operation enters the queue and when it can be observed by a consumer. Second, a contract like the one in Figure 5 could end up aborting all requests if even a single one experienced a storage conflict. We address both problems by slightly relaxing snapshot isolation. When a node assembles a new block to propose, a few system calls can observe the effects of concurrent transactions in a limited way. These effects nondeterministically depend on transaction scheduling, and are captured and included as *proposer data* in blocks. Nodes validating a proposed block rely on proposer data to avoid any dependence on concurrent transactions, thereby ensuring validated transactions are deterministic and fully commutative.

5.1 Commit Locking

When a request-serializing transaction, such as the queue crank in Figure 5, processes multiple requests, individual requests must be able to fail without aborting the entire transaction. For example, if the queue contains five trade requests and one of them cannot complete (e.g., for insufficient funds), the other four requests should still complete.

Preventing a request from aborting the entire transaction is complicated by the fact that, by default, Thruput buffers all writes and does not initiate resource reservations until the very end of the transaction. By that point, a storage conflict may arise too late to handle programmatically.

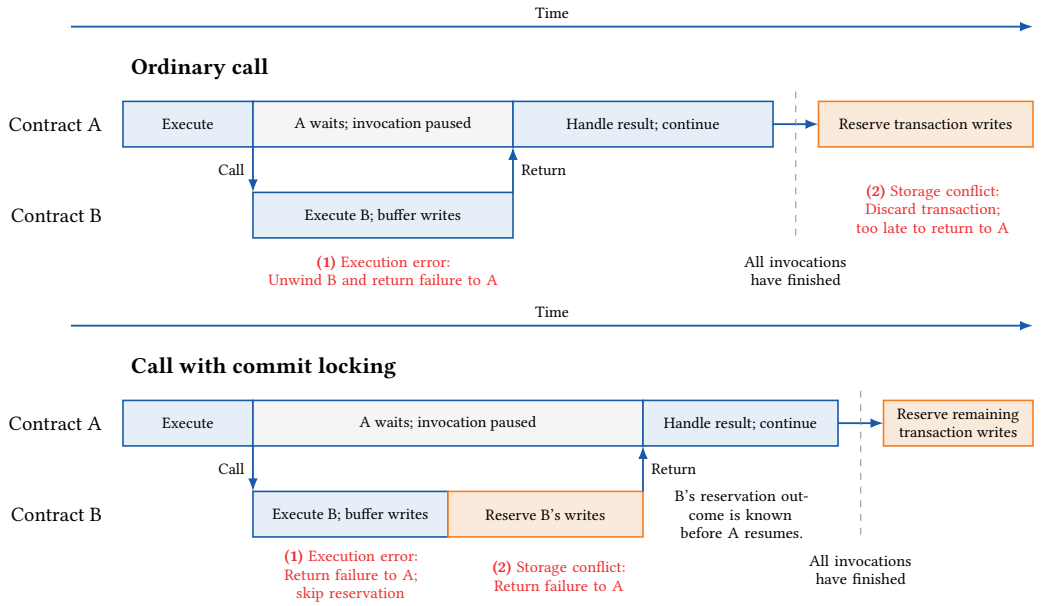


Fig. 6. Commit locking lets the caller handle both execution errors and storage conflicts before continuing. The callee's reservation outcome is logged as one nondeterministic bit of proposer data for replay.

Storage conflicts are even more pernicious in cross-contract calls, as the calling contract may not even know what state the called contract needs to modify or how.

Figure 6 illustrates the problem. If contract A calls contract B, it is possible for contract B to observe an unacceptable state and return an exception (case 1), in which case contract B's modifications are rolled back and contract A can handle the error and proceed as desired. However, if contract B happens to issue a write that results in the transaction being excluded from the block (case 2), contract A will have given up the opportunity to execute entirely.

Thruput lets callers avoid this problem with a mechanism known as *commit locking*. A call made with commit locking performs all of the callee contract's storage reservations as soon as the callee returns, instead of at the end of the transaction. At this point, failed reservations revert all state changes made by the callee, but provide the caller an opportunity to handle the error.

The reservation outcome is recorded in proposer data and replayed by validators. A reported failure does not prove that the reservations could not have succeeded.

5.2 Retrocausality

Naïve queue-based sequencing would delay queued operations by at least one block before they could be visible to a consumer transaction. To avoid this delay, Thruput allows consumers to dequeue operations enqueued in the same block and records such concurrently dequeued messages in proposer data. Validating nodes use proposer data to replay dequeue operations deterministically even for messages that have not yet been enqueued.

Queues are *retrocausal* in that an enqueue operation can effectively go back in time to cause a previously validated dequeue operation to happen. Queues might even produce circular causality, as illustrated in Figure 7. In practice, the default Thruput software cannot produce circular causality, but that is not enforced when validating a block proposal. Validators simply enforce that each

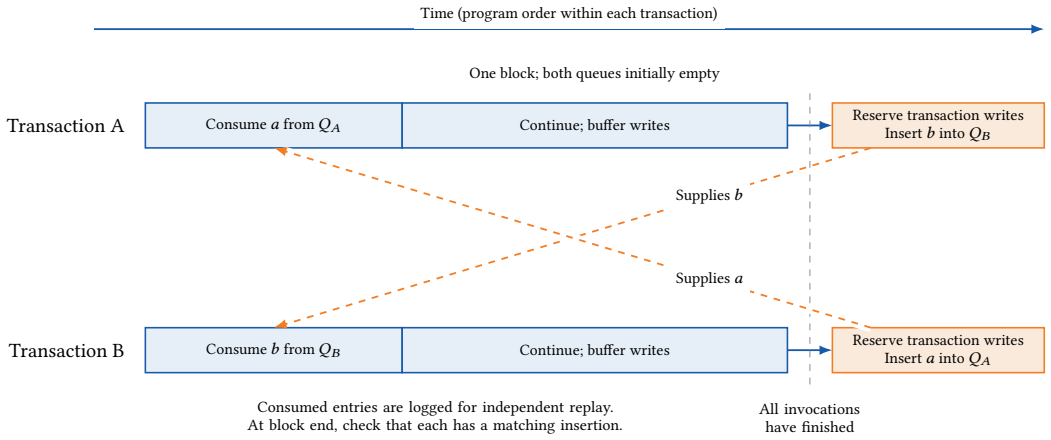


Fig. 7. A circular queue dependency allowed in Thruput. No causal execution order is required.

message is dequeued at most once and that any dequeued message must be enqueued before the end of the current block.

6 Roadmap: External Calls

Section 5 highlights the flexibility that a small amount of nondeterministic data recorded in a block proposal provides. This flexibility, combined with Thruput's unordered execution, can give Thruput contracts a capability not viable on any other blockchain. Contracts can make **remote procedure calls** to services outside of the blockchain, all within a single, atomic Thruput transaction. To be clear, the infrastructure to support this functionality is not yet implemented. The research prototype [77] had a limited form of this capability, but Thruput is still working through the exact design for the API to provide to contracts. There are also security concerns.[6]

Suppose that a contract initiates an RPC to some service off of the blockchain. This involves a network roundtrip over the public internet, which might easily take tens or hundreds of milliseconds, and might or might not return at all. On a standard, sequential blockchain, this would be a complete non-starter of a design, as the entire chain would be forced to stall for this single call. However, in Thruput's unordered execution model, while one transaction is awaiting a pending remote call, a node's CPUs can simply work on running other transactions until that RPC finishes (§8.2). And while we cannot rely on an untrusted off-chain service to provide repeatable answers, we can bundle the call's observed response within the nondeterministic data in a block proposal, so that every validating node can replay the RPC without actually making the call.

There are of course limitations in this design. Any service that wishes to reliably talk to the chain must have server(s) with low-latency connections to most if not all Thruput nodes. The RPC responses would need to be self-authenticating (for example, the response could be signed by a key that is preregistered with the caller contract). And each RPC has an inherent maximum timeout, as we cannot wait for a transaction for longer than Thruput waits for each block.

Of course, asynchronous calls to external services can be implemented on today's blockchains across multiple blocks, but not within the same, atomic transaction. The value lies in situations where low-latency access to external systems is extremely valuable. Consider as an example a contract trading with a market-maker. The contract sends a request to trade to the market-maker,

[6]We do not want Thruput to act as a DoS amplification vector on other services.

who responds with an offered price. On a traditional blockchain, that response (a separate blockchain transaction) might be substantially delayed (by congestion on chain or by normal consensus or network broadcast delays, for example), and the contract has to be written to tolerate a fair amount of delay. This gives the market-maker a so-called "free option" to wait and receive more information about the price of the relevant assets before acting [44, 61, 100, 101]. Thruput, by contrast, can cut that delay down to a single network roundtrip, making that option as small as possible.

A contract could make queries to several market-makers, and trade with the best offered price quote. One could also query oracle data as needed within one transaction, instead of forcing the data provider to post an oracle update (for every possible data feed) in every block. A bank could even allow a transaction to query its KYC/AML processes, approving transfers of regulated assets without posting any private data on-chain.

7 Addresses and Authorization

Computation on Thruput is so widely available that we have no need for so-called "externally owned accounts"; instead, every user interacts with the chain via a smart-wallet contract. Each address can register one or more public keys, which Thruput uses to authorize transactions that start with an invocation on that address. This comes with one exception. To streamline onboarding new users, we allow public keys to interact with the chain via implicit smart wallets, each controlled by its corresponding key (§12.1.1). Those wallets cannot be invoked by other contracts, but are not otherwise distinguishable from any other address, and we plan to add functionality to allow those implicit wallets to upgrade in place to regular smart wallets.

This design has the added benefit of not locking Thruput into any one signature scheme. Given that fault-tolerant quantum computers are plausibly on the horizon and that artificial intelligence is greatly assisting in cryptanalysis [65, 91], it would be imprudent to launch a new blockchain without making it trivial for users to switch between cryptographic schemes. Thruput lets users keep their existing addresses and just register additional keys (and revoke old ones). Every part of the system that interacts with a public key or signature is abstracted over the cryptographic scheme, so contracts generally do not need to think about what schemes are available.

Thruput currently supports Ed25519 keys (for backwards compatibility with existing wallets) and ML-DSA-65 (FIPS 204), but we plan to update this list as additional schemes are standardized and studied.

8 Transaction Execution

As described in §2.1, each transaction runs a smart contract invocation, reads from the database snapshot, and accumulates writes, which are then applied through a two-phase commit process. Thruput is designed to run every transaction in a block in parallel, using as many CPU cores as are available.

By eliminating the ordering between transactions in a block, Thruput ensures that the behavior of one transaction cannot depend on the behavior of a concurrent transaction (with the exceptions in §5). As such, threads in the transaction executor (Figure 2, component E) that are concurrently running transactions have almost no need to communicate or synchronize with one another; the only meaningful shared data structure is a cache of snapshot values loaded from the database. This property is also what allows Thruput to scale easily over multiple physical machines (which would otherwise make cross-thread synchronization much slower).

8.1 RISC-V Interpreter

Thruput runs each smart contract within a RISC-V interpreter. While any small virtual machine will fit in Thruput's parallel model (the Groundhog research project [77] experimented with a wide

range of WebAssembly runtimes, as well as custom sandboxing techniques [103]), a lightweight interpreter provides two key properties.

First, and most important, is that each interpreter instance is a lightweight data structure that requires no system calls to create. Just-in-time (JIT) compiled code generally runs faster on straight-line execution, but requires one or more `mprotect` system calls to change the executable bit on the compiled code. Overall throughput drops precipitously when this system call is invoked concurrently by many threads [103].[7]

And second, implementing a RISC-V interpreter is simple. Thruput uses a direct-threaded [14] interpreter for the `riscv32-im` instruction set, with instruction fusion; nevertheless, the entire interpreter takes just a few thousand lines of code. This makes auditing the code and gaining confidence in its correctness much easier. Furthermore, translating a serialized code artifact into an executable data structure requires only one pass over the instructions to parse them into an array of function pointers.

8.2 Latency Hiding with Coroutines

Lightweight interpreters also integrate easily as *coroutines*. Each CPU core might be midway through running several transactions at any one time. When a transaction waits for a database read, its CPU pauses that transaction and immediately begins work on another runnable transaction. The latency of accesses to the database is therefore hidden, and CPUs do not waste time idling for database responses (Figure 8).

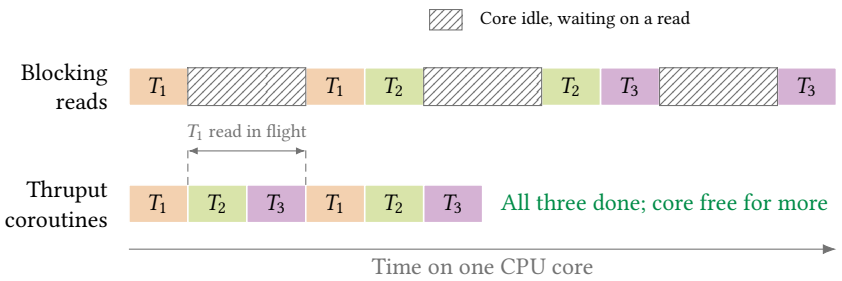


Fig. 8. Hiding read latency on one CPU core.

9 RamDB: Shared State for Distributed Execution

Storage must scale the way execution does (§8), in both capacity and performance, without requiring contracts to know where their state is stored or to declare which keys they will access (§1). Existing chains cannot offer this, because sequential execution puts every access to contended state on one serial path, and they pay for it in one of two ways. The first keeps state in an authenticated structure such as Ethereum's Merkle Patricia trie, where one access costs $O(\log n)$ storage operations and the root is recomputed for every block [37, 56, 97]. The second, Solana's, stores untyped byte-array accounts and requires every transaction to name the accounts it reads and writes [87, 102]. Non-conflicting transactions then run in parallel, but any two transactions that write the same account are serialized, even if they modify different bytes [8]. Developers obtain parallelism for uncontended operations by spreading state across derived accounts under account-size, transaction-size, and

[7] Our internal benchmarks lost around 90% of our performance on an early prototype using a just-in-time WebAssembly compiler, when a patch bump to a crate added an `mprotect` call on each transaction [22].

parallelism limits, and by keeping bulk state off-chain behind a hash [85, 86]. That is a high bar to set when storage itself is not the scarce resource.

9.1 Storage under unordered execution

Thruput's block-level semantics (§2) allow its storage layer to be unaware of transactions. Reads come from the previous block's immutable snapshot (§2), so executors cache them freely and repeated reads of hot keys never reach the store. The contract-state service (`hog_db`) coalesces the block's typed modifications per key as they arrive, and at block close it resolves each modified key to one value. The store then applies the block as one parallel batch of ordinary key-value writes made durable at a single synchronization point. Hence, there is no overlay for reads to consult, no journal to unwind, and no write order to preserve. The block is the unit of durability, as an epoch is in group-commit databases [95], but with no order inside it. Because execution is unordered, a slow store access stalls one transaction rather than the chain (§8.2). Therefore, the store can run on machines separate from execution and be spread across several machines.

9.2 The storage engine

RamDB is a new key-value store, built from scratch for Thruput around this read-phase, write-phase, commit cycle. It keeps all state in DRAM and uses the SSD only as a recovery log. RamDB manages memory as a log, using the design of RAMCloud's log-structured memory [68, 82]: per-thread allocation heads append items into fixed-size segments, a hash table with partitioned locks locates them, overwrites leave tombstones, and a parallel compactor copies live items out of sparse segments. RamDB makes updates durable by appending them to a write-ahead log: writers fill one of many I/O flush buffers chosen by key hash, which avoids a single serialization point while still linearizing writes to each key [45]. A background thread streams full buffers to one on-disk log that is forced once per block. Snapshots are written concurrently with execution. An item is copied into the snapshot just before it is overwritten, so snapshots are consistent without pausing writers or multi-versioning. Recovery streams the snapshot and log back into DRAM and rebuilds just the hash table.

On one 64-core server, RamDB serves over 200 million lookups per second on 64 threads and peaks above 350 million with simultaneous multithreading, with lookup throughput growing almost linearly in the number of cores. It sustains tens of millions of durable inserts, overwrites, or deletes per second, bounded by the roughly 3 GB/s its single log-writer thread streams to the SSD. Under a write-heavy workload in which every update reallocates its item, the store retains about three quarters of its write throughput at a 95% memory-utilization target compared to when run at low memory utilization, showing that memory compaction costs are modest even at high load. High utilization is what makes keeping all state in DRAM cost effective; live data dominates the memory rather than fragmentation.

The service commits to the state in each block header with a homomorphic multiset hash, LtHash [15, 55], as Solana now does [76]. Each item contributes a hash of its key, type, and value to a vector sum that threads accumulate in any order. Thruput relies on the sum in three further ways. First, it avoids an authenticated tree's write amplification; second, it lets a recovering node verify its state on all cores in parallel; and third, because the commitment to a set of items is the sum of the commitments to any partition of it, it lets each machine of a partitioned store verify its own part. The hash attests that a node's state is the state consensus agreed on, not that transactions were valid, which execution and reservation checking enforce (§2.1).

9.3 Scaling the store across machines

We are in the process of extending Thruput's store to spread it across machines, each holding a partition of the key space in its own RamDB instance. When sharding, each key is assigned to a partition by the address that owns it. Executors route reads and reservations to the owning partition, and because repeated reads are served from executor caches, the partition that owns a hot key receives its updates but not its reads. The partitioning is invisible to contracts, transactions, and clients: there are no cross-partition transactions and no placement decisions, because the only operation that spans partitions is the commit of a block. This differs from sharded chains, which partition the chain itself (§1). The closest design, Pilotfish [52], distributes a Sui validator's state across servers but must track every cross-machine dependency during execution because it preserves ordered execution. Each partition verifies its own part of the state against its own partial commitment, and moving keys between partitions can subtract from one partial and add to another, so partitions can be rebalanced while the chain keeps committing to the whole state. A Merkle-rooted store would need a tree per partition and a rebuild whenever keys moved.

Each partition logs its own block commits, but no one needs to record a cross-partition commit decision as is typical in two-phase commit [42], because the chain itself records which blocks are committed. That keeps recovery simple. Each RamDB commits with one synced write, so it restarts at a block boundary. A validator whose partitions restart at different heights can roll them back to the last common height, which RamDB supports by reopening a store at an earlier point in its log. Then it can re-execute the decided blocks from the chain as any lagging node does. No partition needs to write a prepare record or wait on another party, so the blocking failure of classical two-phase commit does not arise.

In sum, contracts are presented with an ordinary typed key-value store without requiring declared access sets. The store performs only per-key work at block granularity, and that locality lets it grow by adding machines, lets each partition verify itself, and lets a validator recover from any crash by rolling back to a block boundary and replaying the chain. Storage can scale the way Thruput's execution scales, by adding hardware, without the choice between performance and usability that today's chains impose.

10 Network Coordination

The coordinator handles all communications with peer validators over public networks. It is responsible for disseminating transactions and blocks, as well as performing the consensus protocol. The current coordinator is written in Go and uses SCP [60] for consensus. We are currently rewriting it in Rust, and have developed a variant of Simplex [25, 84] that works in the same Federated Byzantine Agreement (FBA) model as SCP.

Similar to Alpenglow [53], message dissemination employs a structured broadcast system to carry transaction sets (1 GB per 1M txs), consensus values (100 KiB), and transaction set hashes (32 MB per 1M txs). We use Reed-Solomon coding across trees to overcome broken links, combined with a random shuffle within trees to overcome short blips of total network outage (or outage around the tree root). The coordinator leverages FBA quorum slices to choose tree topologies more resilient to DoS attacks, while also balancing measured latency. However, the design and evaluation are ongoing work and beyond the scope of this paper.

11 Preliminary Evaluation

We evaluate how Thruput's execution engine scales within and across machines. Thruput is under active development, and some components are not yet ready for production. Our experiments measure the executor, database, and events aggregator together as part of a multi-machine validator,

as shown in Figure 9. Our benchmarks are available at `https://github.com/thruputchain/whitney`.

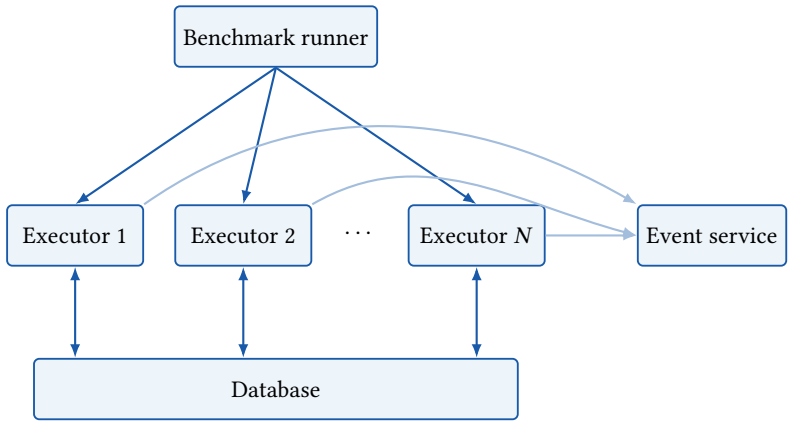


Fig. 9. Services measured by the execution benchmark.

We run the experiments on Amazon EC2 in `us-west-2`, with all machines in the same availability zone and cluster placement group. Each executor runs on an `r8a.16xlarge` instance with 64 AMD Zen 5 virtual cores and 512 GiB of RAM. Three additional machines run the load generator, database, and event service. These machines use `r8a.16xlarge` instances when we vary the number of executor cores and `r8a.24xlarge` instances when we vary the number of executor hosts.

We run each trial three times and report the median. Each transaction (1) invokes a wallet contract, which then (2) invokes a token contract to send a payment. Payments flow between two accounts chosen uniformly at random, with fewer accounts leading to higher per-account contention. All told, each transaction invokes:

- 1 bytestring read (for the wallet's public key)
- 1 replay nonce insertion
- 1 nonnegative integer read[8]
- 2 nonnegative integer writes
- 2 contract loads
- 1 event

The benchmark runner saturates the executors with transactions. Results are a median over three trials, of the median transaction rate over ten blocks, after discarding a warmup block.

11.1 Scaling Within and Across Machines

Figure 10(a) shows how throughput scales as we increase the number of active executor cores on one host from one to 64. At 64 cores, throughput is 51.2× the single-core rate with 100 accounts and 57.7× with 10,000 accounts, demonstrating near-linear scalability. Even a single executor host exceeds one million transactions per second.

The 100-account workload has significantly more per-address contention than with 10,000 accounts, limiting parallelism on blockchains that preserve sequential transaction semantics. Thruput achieves higher throughput on this workload than even with 10,000 accounts as fewer accounts

[8]There is an `nnint_add` system call that allows us to update the recipient balance without reading it.

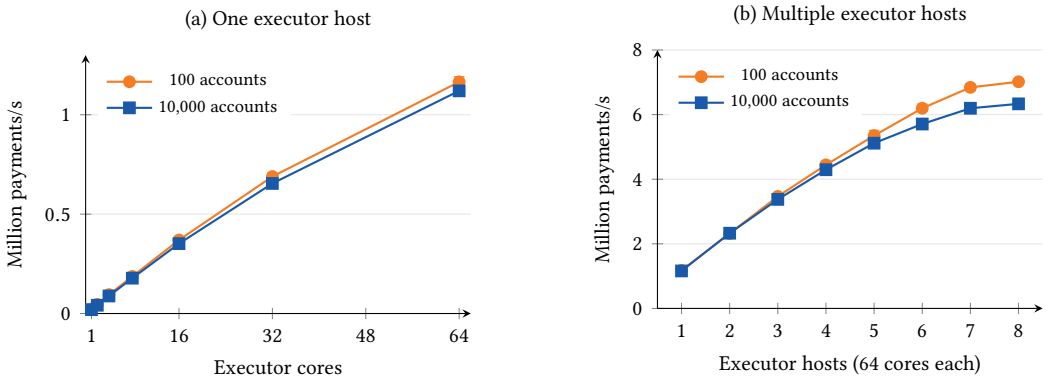


Fig. 10. Execution scaling for ERC20 payments. The single-machine experiment uses 29,000 transactions per active core per block, and the multi-machine experiment uses four million per block.

improve cache locality and reduce the number of updated values written to the database at the end of each block.

We next scale from 1 to 8 executor hosts, each using all 64 cores (Figure 10(b)). With 100 accounts, eight hosts achieve 7.02 million transactions per second, a 5.95× speedup over one host. With 10,000 accounts, throughput reaches 6.33 million transactions per second, a 5.46× speedup.

Scaling tapers off near eight hosts because adding executors does not reduce the fixed overhead for each block. As transaction execution becomes faster, this fixed work accounts for a larger share of block processing time, limiting the benefit of additional executors. Other Thruput subsystems need to be sharded across more machines to maintain Thruput's linear scalability.

11.2 Discussion

These results demonstrate that Thruput can scale transaction throughput by simply increasing hardware resources. Scalability holds even when many transactions update the same accounts. We use a payments benchmark to compare with existing literature, though the absolute transaction rate will vary with other workloads.

This scalability also lets the network support more computation per transaction. More complex contracts than the payment contracts in our benchmark would reduce the transaction rate on the same hardware, but adding executors could offset that cost. Additional capacity could therefore support *more complex smart contracts* as well as more transactions.[9]

12 Capital Expenses, not Operating Expenses

Thruput's raison d'être is to make smart contract execution free. End users should be able to send payments freely, without worrying about any internal details of how Thruput is structured. Nevertheless, there are costs associated with operating Thruput, and there is a cost to keeping data accessible on-chain. Thruput does not aim to be a free alternative to large-scale file storage services like Amazon S3 [5].

Thruput makes these costs into predictable capital costs, to utilize shares of the network's resources, instead of operating expenses, charged on each transaction. Thruput embeds a native token to represent shares of the network resources. These shares can be used, but are not spent.

[9]Benchmarks for this will be published and open-sourced soon.

Our goal is to make it possible in Thruput to operate an application that does not require end users to interact with or own the native token whatsoever. Instead, the operator of a smart contract, such as the issuer of a token, would pay a predictable cost, proportional to the number of holders of the token. Contracts might still charge fees, possibly denominated in the native token, but that is not required.

One of the most frustrating aspects of using today's blockchains is having to acquire and manage a native token just to pay transaction fees. Applications built on today's blockchains appear to be trying to reach this same goal, through increasingly complicated workarounds, such as explicit fee sponsors on Stellar and Solana [48, 88], paymaster contracts under ERC-4337 [21], automatic fee conversion as part of a transaction on Tempo [94], or protocol-level fee exemptions for eligible stablecoin transfers on Sui [92].

We believe that Thruput's model, of simply not charging a per-transaction fee, will prove vastly simpler both to implement and to interact with as a user. We also conjecture that this model could make it easier for autonomous agents to interact with on-chain systems.

12.1 Storage Costs

Storing data on-chain requires locking some amount of the native token. Deleting the data returns the token to the owner. The amount of token locked will be roughly proportional to the size, in bytes, of the data.

As an example, on Thruput, an institution or organization that issues an asset as a token on Thruput would have to lock tokens in proportion to the number of users who are holding that asset (the number of balance entries in the token contract). Importantly, those costs are predictable, given the number of users, and borne by the entity issuing the asset, not the end users. Once those balances exist on-chain, the issuer need not continue to funnel resources into the contract.

To be specific, each contract address has a balance of the native token. Thruput also tracks the number of storage entries owned by that address, and tracks that the cost of those entries is at most the token balance. Transactions that would create entries exceeding the token balance are rejected from block proposals, like any other transaction conflict (§2.1).

Thruput leaves management of the native token for storage costs up to the contract author. For example, the creator of a token could choose to endow the contract address with some amount of the native token, to support an initial base of users. An institution that issues a regulated token might explicitly create each balance entry manually (after, for example, KYC/AML checks) and fund each new entry manually. Conversely, a token operated by a decentralized autonomous organization might require each new holder to provide the native token to fund their balance.

12.1.1 Zero-Storage Wallets. An end user's wallet would ordinarily require deploying a contract on-chain, which would require the user to own some native tokens. Thruput therefore creates a default, implicit wallet contract at addresses with no explicitly deployed contract.

This contract has a basic set of functionality, such as executing multiple calls atomically. The contract is intrinsic to the Thruput runtime, and therefore has no storage cost. A zero-storage wallet is controlled by the public key whose hash is its address. (A nonce included in the hash input allows multiple wallets per public key.) We plan to add additional functionality in the future, such as contract upgrade-in-place capability (although that would require the user to then acquire native tokens for the storage), to turn these wallets into normal smart wallets.

As in §7, we do not want an explicit distinction in Thruput between multiple types of addresses. This design is as close as we are willing to get in Thruput to a distinct notion of an "Externally Owned Account". It cannot be directly invoked by other contracts, but otherwise is observably identical to any other contract in Thruput. Furthermore, like the rest of Thruput, the cryptographic

scheme is not fixed to any one system. Users can choose post-quantum secure wallets from day one.

12.1.2 Roadmap: Sponsored Replay Nonces. Thruput stores a nonce for every committed transaction to prevent transaction replay (§4.4). These logged nonces normally require locking native tokens until the nonces expire (§12.1), which contradicts Thruput's goal of allowing users to transact without holding the native token. We plan to implement a limited nonce functionality for addresses without tokens to store nonces, although we are still debating the exact design. For example, we could allow a very small number of nonces on tokenless addresses and restrict expirations to be near in the future, so that new users could transact at low frequency, but Thruput's costs overall would be limited, and heavy users of Thruput would still need to lock tokens for their nonces. An alternative would be to let contract developers sponsor a limited number of nonces for their users.

12.2 No-Fee Transaction Scheduling

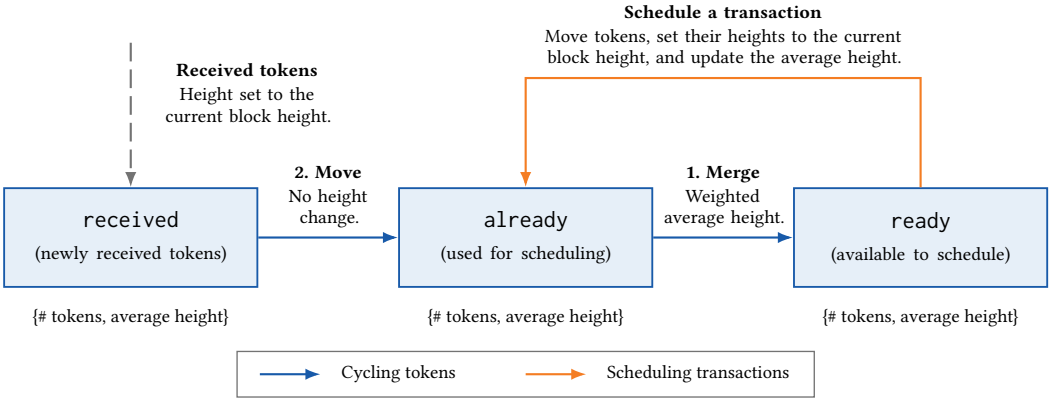
Thruput uses its native token to divide computation when demand exceeds capacity. Addresses with more tokens receive a larger share, but scheduling a transaction does not spend the tokens. The same tokens can schedule future transactions, so they are a capital cost rather than a fee for each transaction. Each validator keeps pending transactions in a mempool, and the mempool's scheduler chooses which transactions to consider for a block proposal (§2.1).

12.2.1 Scheduling Goals. Thruput can add machines to increase capacity, but capacity is always finite. When more transactions arrive than validators can execute, the scheduler decides which ones to consider first for inclusion in a block.

(a) Long-Term Fairness. In the long run, addresses should receive computation in proportion to their token balances. This prevents one user from monopolizing the network by submitting many transactions quickly. For example, if two addresses continuously submit transactions with the same computation limit, an address with twice as many tokens should execute twice as many transactions over time. We also want to minimize incentives to game the scheduling policy, such as by splitting a balance across several addresses and holding some addresses inactive to retain a higher scheduling priority when those addresses later start submitting transactions. Otherwise, users would have an incentive to create additional addresses and submit transfers solely to improve their scheduling priority, consuming storage and computation without doing useful work.

(b) Scheduling without Fees. To share computation fairly over time, the scheduler tracks each token's *age*. The age is the current block height minus the greater of the height at which the token was last used to schedule a transaction and the height at which the token was received in a payment. A token with a higher age receives higher scheduling priority. Scheduling a transaction resets this age to zero for each scheduled token. The tokens remain at the address and can be used to schedule future transactions. Users therefore do not need to replenish their balances just to keep executing transactions. Each transaction still declares a computation limit to bound its computation and determine how many tokens are used for scheduling. A transaction requesting more computation uses more of its address's scheduling priority without incurring a fee.

12.2.2 Conceptual Token Queue. Conceptually, all tokens from addresses with pending transactions are placed into a single queue, where tokens with the greatest age are at the front. Each transaction declares a computation limit that determines how many tokens its address must dedicate to schedule it. The scheduler selects that number of tokens from the address, starting with those closest to the front of the queue. The position of the last selected token determines the transaction's priority, so a transaction whose last token is closer to the front has higher priority. When capacity is limited,



Cycle buckets: The scheduler cycles buckets when `ready` cannot cover the next transaction or cycling improves scheduling priority.
To cycle: First merge `already` into `ready`, then move `received` into the emptied `already` bucket.

Fig. 11. The three token groups for one address. Addresses do not pay tokens to execute transactions. Instead, addresses are prioritized based on the age derived from the average block height stored for their `ready` group.

only the highest-priority transactions are included in the block. Once a block is committed, the scheduler moves the tokens for the included transactions to the back of the queue, but the tokens are not otherwise moved to different addresses.

12.2.3 Practical Approximation. Rather than manage the overhead of tracking each token's position individually, validators approximate the conceptual queue using three token groups per address: tokens available for scheduling (`ready`), tokens recently used for scheduling (`already`), and tokens recently received (`received`). For each group, the validator tracks the average block height at which its tokens were last scheduled or received. This keeps scheduling state fixed in size while preserving the queue's long-term fairness properties. Figure 11 shows these groups and their transitions.

13 Related Work

13.1 Semantic Changes

Thruput is a direct continuation of Groundhog [77]. That work is itself derived from SPEEDEX [79], which is a decentralized exchange using the same model of blocks containing unordered, parallel transactions. It operated an exchange by designing custom, commutative operations for transactions to interact with orderbooks. This approach is modeled on the Scalable Commutativity Rule of Clements et al. [27]. By redesigning kernel interfaces to be commutative, the authors build an operating system that scales extremely well. Other projects, including Canidio and Henneke [23], WalrasSwap [105], and Angstrom [90], settle batches of unordered trades (on sequential blockchains, like Ethereum), albeit for economic and market efficiency reasons, not performance.

Conflict-Free Replicated Data Types [83] are commutative interfaces for modifying shared (distributed) data structures. CRDTs, however, require that any update that is arbitrarily delayed can be reconciled with a local copy of the state. CRDTs can therefore implement an increasing counter, for example, but cannot implement a notion of a token balance, which must increase and decrease and stay nonnegative, without some kind of synchronization. Two replicas might both subtract the last dollar from an account, and the CRDT must reconcile these updates. Balegas et al. [10] split the notion of a balance across a fixed, known set of replicas, and then each replica only admits subtraction to the shared counter that it locally can guarantee will not violate the

nonnegativity constraint. That approach requires every replica to be trusted, and is akin to sharding a balance over many separate networks.

O'Neil [67] proposes an "escrow method" to add concurrency to a database. Transactions acquire fine-grained reservations on aggregate quantities (such as balances of money), so that concurrent transactions can modify them while maintaining invariants. Thruput uses a similar reservation mechanism, but our snapshot read semantics provide deterministic database reads of a predictable snapshot, and the end-of-block synchronization enables primitives like `nnint_set_add`, which are strictly more expressive than a balance add/subtract escrow operation (e.g. §4.2.2).

Related techniques also accelerate execution on sequential blockchains. Garamvölgyi et al. [38] add a "commutative add" operation to the Ethereum Virtual Machine, which lets two transactions that just increment a shared counter not conflict under optimistic concurrency control. Mitenkov et al. [63] construct "Deferred Object" types (including Aptos's aggregators [62]) which modify highly contended items, like counters, in a structured way (i.e., output a counter increment without observing the value). This technique can reduce the frequency of transaction conflicts in optimistic concurrency control on some workloads, but does not guarantee parallelism on potentially adversarial workloads and still pays the overhead of concurrency control.

13.2 Parallelizing Sequential Execution

Accelerating operations on a sequential state machine is a long-running open systems and engineering research problem. Block-STM [39] optimistically executes transactions concurrently on many CPU cores while recording which objects are read and written to, retrying transactions when a speculatively read object changes. This kind of approach scales well when few transactions touch shared state, but cannot perform significantly better than sequential execution under high contention. Furthermore, the overhead of the concurrency control data structures introduces engineering complexity and adds significant performance overhead, especially when using many CPU cores.

Chen et al. [26] speculatively execute Ethereum transactions, and Lin et al. [57] instrument transactions so that an optimistic concurrency control process can retry only parts of a transaction, instead of the entire transaction, in case of conflicts. Zhang et al. [108] combine speculated execution traces with a branch predictor, so that actual execution results are more likely to match a pre-executed trace.

Eve [51] schedules batches of transactions in parallel using a conflict checker. Eve guesses at what transactions do not conflict with each other, executes batches of hopefully-nonconflicting transactions on separate CPUs, and then verifies that no conflicts occurred. Performance relies on this conflict checker; conflict check errors degrade performance (to no better than sequential execution, in the extreme). It is not clear if it is possible to write a conflict checker that is hardened against adversarial smart contract inputs without requiring transactions to specify exactly which keys are read from and written to. Thruput requires no predeclared read/write sets and does not track read/write dependencies to enforce a sequential transaction order. It checks reservations against each value type's constraints (§2.1). FAFO [106] similarly estimates which keys a transaction will write, but discards transactions when those estimates are found to be inaccurate. Like Block-STM, it suffers performance degradation under nontrivial contention. The authors analyze workloads to estimate the maximum achievable parallelism given perfect scheduling, but leave a significant gap between possible and realized gains from parallelism.

Strife [74] splits transactions into nonconflicting clusters, while Bohm [36] assigns a serialization order and only begins executing transactions when all read dependencies are available. Strife requires advance knowledge of read/write sets, while Bohm requires predeclared (or speculated) write sets.

Similar in spirit to Thruput are Aria [59] and LiTM [99], which execute batches of transactions in parallel optimistically, and then retry any that abort due to concurrency conflicts in a subsequent batch. These systems still maintain the abstraction of serializability.

Basil [93] allows clients to transact with a replicated key-value store using an optimistic locking protocol, so that neither malicious clients nor malicious nodes can halt the system. This system does not, however, support smart contracts or guarantee correctness of values written by clients.

Some projects use read-write sets [11, 104] or static analysis [24] to precompute sets of transactions to execute in parallel, include an observed dependency graph in a block proposal [6, 7, 34], or change transaction order to increase parallelism [81], or use combinations of these techniques. Pan et al. [69] add instruction-level parallelism to accelerate execution of individual transactions and design an ASIC for Ethereum bytecode.

13.3 Distributed Execution

Close to our authentication service design is Remora [58], which separates the stateless but compute-heavy parts of a transaction (such as checking a signature) from the (shorter) rest of the transaction that reads and writes state. Like Thruput, Remora executes these compute-heavy operations prior to consensus. Remora also sends the work of executing transactions out to stateless executors, but uses a coordinator to manage the sequencing and object multi-versioning between transactions that read and write to the same objects (and relies on statically known read and write sets to do this). The Remora coordinator also elastically scales the worker machines up and down in response to transaction demand.

Shardines [32] distributes Block-STM over a cluster of machines, using a coordinator to send batches of transactions to executors. Executors must communicate with each other whenever a transaction has a data dependency on a transaction scheduled on another shard, so the role of the coordinator is to minimize those cross-shard transactions.

13.4 Transacting Without Consensus

Another approach to scaling blockchains is to avoid consensus on certain transaction patterns. These systems restrict transactions to specific operations that do not need a global order. Guerraoui et al. [43] show an account can send payments using a Byzantine-fault tolerant broadcast. FastPay [12] implements payments by allowing each validator to separately certify a transfer, which becomes irreversible when a sufficient fraction certify it. Zef [13] extends the approach to anonymous, private payments. These systems allow a faulty client to lock their own objects but not those of others. Blackshear et al. [16] extend the approach to a consensus-free fast path for objects owned by one account (in a system where objects have explicit ownership), combined with consensus for transactions (such as more complex smart contract workloads) that manipulate shared objects.

Stingray [89] extends this approach to concurrent operations on bounded counters, using a primitive like Thruput's nonnegative integers to allow concurrent spends from an account. Each validator can individually certify spending a fraction of the account, and uses a consensus-based protocol to recover when an account balance becomes stuck. FirstSwap [33] extends this approach to trading on an automated market-maker.

Pod [4] implements a fault-tolerant timestamping service. Clients combine logs from each Pod node to derive transaction confirmation and a relative ordering between transactions (i.e., by median confirmation timestamp). The design outlined in the paper does not execute transactions itself or maintain shared state, but instead relies on clients to manage their own application state. However, as of this writing, the current Pod network maintains state per validator, like FastPay, but with an account recovery primitive in case an account gets locked [70]. As an example, Pod builds

an orderbook application [71] in which users send transactions to post orders but an external solver computes matchings.

13.5 Sharding and Layer-2 Chains

A common approach to scaling a blockchain is to create more blockchains, under various names including Sharding [66, 98, 107] and Layer-2 rollups [50]. Each component runs separately, but state, like contracts and user balances, is subdivided among the components. This subdivision mandates complex cross-shard bridges or synchronization protocols [31], which are more expensive and slower than regular transactions. User experience is similarly made more complicated than we believe it need be, as a user must understand the internal architecture of the system well enough to understand on which shard their balance is stored.

Payment channels, such as those used by the Lightning Network [73], scale payments by locking funds on-chain and allowing participants to update allocation of those funds off-chain (and only settling on-chain as necessary). These channels can be linked together into networks, creating tradeoffs between network topology, capital costs, on-chain costs, and routing complexity [9, 18, 30, 40, 41, 78]. Plasma [72] extends this idea towards "child chains" with fraud proofs.

14 Conclusion

Thruput makes smart-contract computation horizontally scalable across arbitrarily many servers within each blockchain validator. The key insight is designing an API that makes all transactions in a block fully commutative while remaining expressive enough to implement most common smart-contract functions. With a few limited exceptions, transactions in a block do not see each other's effects, but rather all see the state of the ledger as of the previous block close. A non-negative integer abstraction outputs commutative deltas, allowing a large number of transactions to add or subtract from the same account balances. A retrocausal queue abstraction allows transactions that need sequential request processing to implement sequencer gadgets that run on one CPU without imposing a global transaction order. The execution benchmark processes over 7M payments per second on 8 executor servers, with separate load-generator, database, and event-service machines (§11). This measures execution within one validator, excluding network dissemination and consensus.

Thruput introduces a starvation-free transaction scheduling model in which token balances convey proportional scheduling weight. Under contention, the age of tokens that cannot be scheduled provides a useful signal to determine whether or not to increase capacity. Because Thruput can scale to very high transaction rates, it can entirely eliminate transaction fees and make smart contract execution free.

Acknowledgments

Thruput builds on ideas and results from Groundhog [77], a research prototype built by a few of the authors at Stanford University as part of Stanford's Future of Digital Currency Initiative (FDCI). Adam Belay made early contributions to the architecture and `hog_db` as we began the implementation of Thruput.

References

[1] Aave. Aave V3 technical paper. Technical paper, January 2022. URL `https://github.com/aave/aave-v3-core/raw/refs/heads/master/techpaper/Aave_V3_Technical_Paper.pdf`. Accessed: 2026-10-29.

[2] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 Core. `https://app.uniswap.org/whitepaper.pdf`, 2020.

[3] Elvira Albert, Samir Genaim, Daniel Kirchner, and Enrique Martin-Martin. Formally verified EVM block-optimizations. In *International Conference on Computer Aided Verification*, pages 176–189. Springer, 2023.

[4] Orestis Alpos, Bernardo David, Jakov Mitrovski, Odysseas Sofikitis, and Dionysis Zindros. Pod: An optimal-latency, censorship-free, and accountable generalized consensus layer. *arXiv preprint arXiv:2501.14931*, 2025.

[5] Amazon Web Services. Amazon S3: Cloud object storage. `https://aws.amazon.com/s3/`, n.d. Accessed: 2026-10-25.

[6] Parwat Singh Anjana, Sweta Kumari, Sathya Peri, Sachin Rathor, and Archit Somani. An efficient framework for optimistic concurrent execution of smart contracts. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pages 83–92. IEEE, 2019.

[7] Parwat Singh Anjana, Hagit Attiya, Sweta Kumari, Sathya Peri, and Archit Somani. Efficient concurrent execution of smart contracts in blockchains using object-based transactional memory. In *International Conference on Networked Systems*, pages 77–93. Springer, 2020.

[8] Anza. Runtime on a Solana validator. Agave Validator Documentation, 2025. URL `https://docs.anza.xyz/validator/runtime`. Accessed: 2026-10-22.

[9] Vivek Bagaria, Joachim Neu, and David Tse. Boomerang: Redundancy improves latency and throughput in payment-channel networks. In *International Conference on Financial Cryptography and Data Security*, pages 304–324. Springer, 2020.

[10] Valter Balegas, Diogo Serra, Sérgio Duarte, Carla Ferreira, Marc Shapiro, Rodrigo Rodrigues, and Nuno Preguiça. Extending eventually consistent cloud databases for enforcing numeric invariants. In *2015 IEEE 34th Symposium on Reliable Distributed Systems (SRDS)*, pages 31–36. IEEE, 2015.

[11] Massimo Bartoletti, Letterio Galletta, and Maurizio Murgia. A true concurrent model of smart contracts executions. In *International Conference on Coordination Languages and Models*, pages 243–260. Springer, 2020.

[12] Mathieu Baudet, George Danezis, and Alberto Sonnino. FastPay: High-performance byzantine fault tolerant settlement, 2020. URL `https://arxiv.org/abs/2003.11506`.

[13] Mathieu Baudet, Alberto Sonnino, Mahimna Kelkar, and George Danezis. Zef: Low-latency, scalable, private payments, 2022. URL `https://arxiv.org/abs/2201.05671`.

[14] James R Bell. Threaded code. *Communications of the ACM*, 16(6):370–372, 1973.

[15] Mihir Bellare and Daniele Micciancio. A new paradigm for collision-free hashing: Incrementality at reduced cost. In *Advances in Cryptology — EUROCRYPT '97*, volume 1233 of *Lecture Notes in Computer Science*, pages 163–192. Springer, 1997. doi: 10.1007/3-540-69053-0_13.

[16] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, Brandon Williams, and Lu Zhang. Sui Lutris: A blockchain combining broadcast and consensus, 2023. URL `https://arxiv.org/abs/2310.18042`.

[17] Blend. Blend whitepaper. Blend Protocol Documentation, 2026. URL `https://docs.blend.capital/blend-whitepaper`. Last updated: 2026-01-09. V1 whitepaper with V2 differences. Accessed: 2026-10-29.

[18] Simina Brânzei, Erel Segal-Halevi, and Aviv Zohar. How to charge Lightning: The economics of Bitcoin transaction channels. In *2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 1–8. IEEE, 2022.

[19] Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

[20] Eric B Budish, Peter Cramton, Albert S Kyle, and Jeongmin Lee. Flow trading. Working Paper 2022-82, University of Chicago, Becker Friedman Institute for Economics, 2022.

[21] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. ERC-4337: Account abstraction using alt mempool. Ethereum Improvement Proposals, September 2021. URL `https://eips.ethereum.org/EIPS/eip-4337`. Created 2021-09-29. Accessed: 2026-10-25.

[22] Bytecode Alliance. Wasmtime has data leakage between pooling allocator instances. GitHub Security Advisory GHSA-6wgr-89rj-399p (CVE-2026-34988), April 2026. URL `https://github.com/advisories/GHSA-6wgr-89rj-399p`. Published April 9, 2026.

[23] Andrea Canidio and Felix Henneke. Fair combinatorial auctions: Endogenous best execution in blockchain trade-intent markets, 2026. URL `https://arxiv.org/abs/2408.12225`.

[24] Atefeh Zareh Chahoki, Maurice Herlihy, and Marco Roveri. Conthereum: Concurrent Ethereum optimized transaction scheduling for multi-core execution. In *2025 7th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, pages 1–10. IEEE, 2025.

[25] Benjamin Y Chan and Rafael Pass. Simplex consensus: A simple and fast consensus protocol. In *Theory of Cryptography Conference*, pages 452–479. Springer, 2023.

[26] Yang Chen, Zhongxin Guo, Runhuai Li, Shuo Chen, Lidong Zhou, Yajin Zhou, and Xian Zhang. Forerunner: Constraint-based speculative transaction execution for Ethereum. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 570–587, 2021.

[27] Austin T Clements, M Frans Kaashoek, Nickolai Zeldovich, Robert T Morris, and Eddie Kohler. The scalable commutativity rule: Designing scalable software for multicore processors. *ACM Transactions on Computer Systems*

(TOCS), 32(4):1–47, 2015.

[28] Compound. Compound III docs. `https://docs.compound.xyz/`, 2022.

[29] Compound. Comet: Interest accrual implementation. `https://github.com/compound-finance/comet/blob/f766f51583c23acc33b2a7824654ef2029a96804/contracts/CometWithExtendedAssetList.sol#L243-L255`, June 2026. Function `accrueInternal`, revision `f766f51583c2`.

[30] Pranav Dandekar, Ashish Goel, Ramesh Govindan, and Ian Post. Liquidity in credit networks: A little trust goes a long way. In *Proceedings of the 12th ACM conference on Electronic commerce*, pages 147–156, 2011.

[31] Sourav Das, Vinith Krishnan, and Ling Ren. Efficient cross-shard transaction execution in sharded blockchains. *arXiv preprint arXiv:2007.14521*, 2020.

[32] Manu Dhundi, Sital Kedia, Igor Kabiljo, Zhoujun Ma, and Jan Olkowski. Shardines: Aptos' sharded execution engine blazes to 1M TPS. Aptos Labs, Medium, February 2025. URL `https://medium.com/aptoslabs/shardines-aptos-sharded-execution-engine-blazes-to-1m-tps-71c5f9b8bf60`.

[33] William Diamond, Fatima Elsheimy, Dahlia Malkhi, Geoff Ramseyer, and Srivatsan Sridhar. FirstSwap: Pre-consensus trade approvals for decentralized market design. *Available at SSRN 7652139*, 2026.

[34] Thomas Dickerson, Paul Gazzillo, Maurice Herlihy, and Eric Koskinen. Adding concurrency to smart contracts. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, pages 303–312, 2017.

[35] Leonardo Donatacci. How to write Solana programs with sBPF assembly. `https://www.helius.dev/blog/sbpf-assembly`, September 2025. Published 4 September 2025. Accessed: 2026-10-18.

[36] Jose M Faleiro and Daniel J Abadi. Rethinking serializable multiversion concurrency control. *arXiv preprint arXiv:1412.2324*, 2014.

[37] Zhenhuan Gao, Yuxuan Hu, and Qinfan Wu. Jellyfish Merkle tree. Diem Association technical paper, January 2021. URL `https://developers.diem.com/papers/jellyfish-merkle-tree/2021-01-14.pdf`.

[38] Péter Garamvölgyi, Yuxi Liu, Dong Zhou, Fan Long, and Ming Wu. Utilizing parallelism in smart contracts on decentralized blockchains by taming application-inherent conflicts. In *Proceedings of the 44th international conference on software engineering*, pages 2315–2326, 2022.

[39] Rati Gelashvili, Alexander Spiegelman, Zhuolun Xiang, George Danezis, Zekun Li, Dahlia Malkhi, Yu Xia, and Runtian Zhou. Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 232–244, 2023.

[40] Ashish Goel and Geoffrey Ramseyer. Continuous credit networks and layer 2 blockchains: Monotonicity and sampling. In *Proceedings of the 21st ACM Conference on Economics and Computation*, pages 613–635, 2020.

[41] Ashish Goel, Sanjeev Khanna, Sharath Raghvendra, and Hongyang Zhang. Connectivity in random forests and credit networks. In *Proceedings of the twenty-sixth annual ACM-SIAM symposium on Discrete algorithms*, pages 2037–2048. SIAM, 2014.

[42] Jim Gray and Leslie Lamport. Consensus on transaction commit. *ACM Transactions on Database Systems*, 31(1): 133–160, 2006. doi: 10.1145/1132863.1132867.

[43] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, and Dragos-Adrian Seredinschi. The consensus number of a cryptocurrency (extended version), 2019. URL `https://arxiv.org/abs/1906.05574`.

[44] Runchao Han, Haoyu Lin, and Jiangshan Yu. On the optionality and fairness of atomic swaps. In *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, pages 62–75, 2019.

[45] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. doi: 10.1145/78969.78972.

[46] Ivan Indriawan, Roberto Pascual, and Andriy Shkilko. On the effects of continuous trading. *Available at SSRN 3707154*, 2020.

[47] Ravi Jagannathan. On frequent batch auctions for stocks. *Journal of Financial Econometrics*, 20(1):1–17, 2022.

[48] Jonathan Jove. How we simplified fees on the Stellar network. Stellar Development Foundation, May 2020. URL `https://stellar.org/blog/developers/how-we-simplified-fees-on-the-stellar-network`. Accessed: 2026-10-25.

[49] Jupiter. Technical reference. Jupiter Perps Documentation, 2026. URL `https://docs.jup.ag/user-docs/trade/perps/technical-reference#request-fulfillment-model`. Last updated: 2026-10-13. Sections: Request Fulfillment Model and Keepers. Accessed: 2026-10-31.

[50] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S Matthew Weinberg, and Edward W Felten. Arbitrum: Scalable, private smart contracts. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1353–1370, 2018.

[51] Manos Kapritsos, Yang Wang, Vivien Quema, Allen Clement, Lorenzo Alvisi, and Mike Dahlin. All about Eve: Execute-Verify replication for Multi-Core servers. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, pages 237–250, 2012.

[52] Quentin Kniep, Lefteris Kokoris-Kogias, Alberto Sonnino, Igor Zablotchi, and Nuda Zhang. Pilotfish: Distributed execution for scalable blockchains. In *Financial Cryptography and Data Security — 29th International Conference, FC*

2025, Revised Selected Papers, Part I, volume 15751 of *Lecture Notes in Computer Science*, pages 287–306. Springer, 2025. doi: 10.1007/978-3-032-07024-1_17.

[53] Quentin Kniep, Jakub Sliwinski, and Roger Wattenhofer. Solana Alpenglow Consensus, 2025. URL https://quentinkniep.com/research/solana-alpenglow-consensus.html. White paper.

[54] Yi-Tsung Lee, Roberto Riccò, and Kai Wang. Frequent batch auctions vs. continuous trading: Evidence from Taiwan. *Journal of Financial Markets*, page 101082, 2026.

[55] Kevin Lewi, Wonho Kim, Ilya Maykov, and Stephen Weis. Securing update propagation with homomorphic hashing. Cryptology ePrint Archive, Paper 2019/227, 2019. URL https://eprint.iacr.org/2019/227.

[56] Chenxing Li, Sidi Mohamed Beillahi, Guang Yang, Ming Wu, Wei Xu, and Fan Long. LVMT: An efficient authenticated storage for blockchain. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 135–153, Boston, MA, 2023. USENIX Association. URL https://www.usenix.org/conference/osdi23/presentation/li-chenxing.

[57] Haoran Lin, Hang Feng, Yajin Zhou, and Lei Wu. ParallelEVM: Operation-level concurrent transaction execution for EVM-compatible blockchains. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 211–225, 2025.

[58] Zhengqing Liu, Alberto Sonnino, Igor Zablotchi, Eleftherios Kokoris Kogias, and Marios Kogias. Remora: Scale-out deterministic execution for smart contracts, 2026. URL https://arxiv.org/abs/2607.02817.

[59] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. Aria: a fast and practical deterministic OLTP database. *Proceedings of the VLDB Endowment*, 13(11):2047–2060, 2020. doi: 10.14778/3407790.3407808. URL https://www.vldb.org/pvldb/vol13/p2047-lu.pdf.

[60] David Mazières. The Stellar consensus protocol: A federated model for internet-level consensus, April 2015. URL https://www.stellar.org/papers/stellar-consensus-protocol.pdf.

[61] Bruno Mazorra, Burak Öz, Christoph Schlegel, and Fei Wu. The free option problem of ePBS. *arXiv preprint arXiv:2509.24849*, 2025.

[62] George Mitenkov, Satya Vusirikala, Rati Gelashvili, and Igor Kabiljo. AIP-47: Aggregators V2. https://github.com/aptos-foundation/AIPs/blob/main/aips/aip-047-aggregators-v2.md, August 2023. Status: Accepted. Standard (Core/Framework). Accessed: 2026-10-20.

[63] George Mitenkov, Igor Kabiljo, Zekun Li, Alexander Spiegelman, Satyanarayana Vusirikala, Zhuolun Xiang, Aleksandar Zlateski, Nuno P Lopes, and Rati Gelashvili. Deferred objects to enhance smart contract programming with optimistic parallel execution. *arXiv preprint arXiv:2405.06117*, 2024.

[64] Stephen Monn and Bikem Bengisu. ERC-7521: General intents for smart contract wallets. https://eips.ethereum.org/EIPS/eip-7521, September 2023. Draft, Standards Track: ERC. Created 2023-09-19. Accessed: 2026-10-19.

[65] Milad Nasr and Nicholas Carlini. Cryptanalysis of 7-round AES via the algebraic structure of its S-box. Technical report, Anthropic, 2026.

[66] NEAR Team. NEAR launches Nightshade sharding, paving the way for mass adoption. https://near.org/blog/near-launches-nightshade-sharding-paving-the-way-for-mass-adoption/, November 2021. Accessed 10/18/2022.

[67] Patrick E O'Neil. The escrow transactional method. *ACM Transactions on Database Systems (TODS)*, 11(4):405–430, 1986.

[68] John Ousterhout, Arjun Gopalan, Ashish Gupta, Ankita Kejriwal, Collin Lee, Behnam Montazeri, Diego Ongaro, Seo Jin Park, Henry Qin, Mendel Rosenblum, Stephen Rumble, Ryan Stutsman, and Stephen Yang. The RAMCloud storage system. *ACM Transactions on Computer Systems*, 33(3):7:1–7:55, 2015. doi: 10.1145/2806887.

[69] Rui Pan, Chubo Liu, Guoqing Xiao, Mingxing Duan, Keqin Li, and Kenli Li. An algorithm and architecture co-design for accelerating smart contracts in blockchain. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–13, 2023.

[70] Pod Network. Local ordering. Pod Network Documentation (v2), 2026. URL https://docs.v2.pod.network/documentation/core/local-ordering. Last updated: 2026-02-16. Accessed: 2026-11-02.

[71] Pod Network. Order book. Pod Network Documentation (v2), 2026. URL https://docs.v2.pod.network/documentation/markets/orderbook. Last updated: 2026-02-16. Accessed: 2026-11-02.

[72] Joseph Poon and Vitalik Buterin. Plasma: Scalable autonomous smart contracts, August 2017. URL https://www.plasma.io/plasma.pdf. Working draft, August 11, 2017.

[73] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning Network: Scalable off-chain instant payments, 2016. URL https://lightning.network/lightning-network-paper.pdf. Draft version 0.5.9.2.

[74] Guna Prasaad, Alvin Cheung, and Dan Suciu. Handling highly contended OLTP workloads using fast dynamic partitioning. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 527–542, 2020.

[75] Project Serum. Serum Core, 2021. URL https://docs.projectserum.com/appendix/serum-core. Last updated: 2021-09-29. Accessed: 2026-10-31.

[76] Brooks Prumo, Emanuele Cesena, Josh Siegel, and Sam Kim. SIMD-0215: Homomorphic hashing of account state. Solana Improvement Documents, December 2024. URL https://github.com/solana-foundation/solana-improvement-documents/blob/main/proposals/0215-accounts-lattice-hash.md. Accessed: 2026-10-22.

[77] Geoffrey Ramseyer and David Mazières. Groundhog: Linearly-scalable smart contracting via commutative transaction semantics. arXiv preprint arXiv:2404.03201, 2024. URL https://arxiv.org/abs/2404.03201.

[78] Geoffrey Ramseyer, Ashish Goel, and David Mazières. Liquidity in credit networks with constrained agents. In *Proceedings of The Web Conference 2020*, pages 2099–2108, 2020.

[79] Geoffrey Ramseyer, Ashish Goel, and David Mazières. SPEEDEX: A scalable, parallelizable, and economically efficient decentralized EXchange. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 849–875, 2023.

[80] RareSkills. The RareSkills book of Solidity gas optimization: 80+ tips. https://rareskills.io/post/gas-optimization, 2023. Accessed: 2026-10-18.

[81] Pingcheng Ruan, Dumitrel Loghin, Quang-Trung Ta, Meihui Zhang, Gang Chen, and Beng Chin Ooi. A transactional perspective on execute-order-validate blockchains. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 543–557, 2020.

[82] Stephen M. Rumble, Ankita Kejriwal, and John Ousterhout. Log-structured memory for DRAM-based storage. In *12th USENIX Conference on File and Storage Technologies (FAST 14)*, pages 1–16, Santa Clara, CA, 2014. USENIX Association. URL https://www.usenix.org/conference/fast14/technical-sessions/presentation/rumble.

[83] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*, pages 386–400. Springer, 2011.

[84] Victor Shoup. Sing a song of Simplex. *Cryptology ePrint Archive*, 2023.

[85] Solana Foundation. State compression. Solana Documentation, 2025. URL https://solana.com/docs/advanced/state-compression.

[86] Solana Foundation. Accounts. Solana Documentation, 2026. URL https://solana.com/docs/core/accounts. Accessed: 2026-10-22.

[87] Solana Foundation. Transactions. Solana Documentation, 2026. URL https://solana.com/docs/core/transactions. Accessed: 2026-10-22.

[88] Solana Foundation. Fee abstraction. Solana Documentation, 2026. URL https://solana.com/docs/payments/send-payments/payment-processing/fee-abstraction. Accessed: 2026-10-25.

[89] Srivatsan Sridhar, Alberto Sonnino, and Lefteris Kokoris-Kogias. Stingray: Fast concurrent transactions without consensus. *arXiv preprint arXiv:2501.06531*, 2025.

[90] Karthik Srinivasan, Ludwig Thouvenin, and Ciamac Moallemi. Angstrom v1 whitepaper. Technical report, Sorella Labs, July 2025. URL https://app.angstrom.xyz/whitepaper-v1.pdf. Accessed: 2026-10-19.

[91] Zygimantas Straznickas and Stephen A Weis. HAWK-n key recovery reduces to SVP in dimension $n/2+1$. *Cryptology ePrint Archive*, 2026.

[92] Sui Foundation. Gasless stablecoin transfers. Sui Documentation, 2026. URL https://docs.sui.io/develop/transaction-payment/gasless-stablecoin-transfers. Last updated: 2026-05-18. Accessed: 2026-10-25.

[93] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. Basil: Breaking up BFT with ACID (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 1–17, 2021.

[94] Tempo. Fee AMM specification. Tempo Protocol Documentation, 2026. URL https://docs.tempo.xyz/protocol/fees/spec-fee-amm. Last updated: 2026-01-20. Accessed: 2026-10-25.

[95] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. Speedy transactions in multicore in-memory databases. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP '13)*, pages 18–32, Farmington, PA, 2013. ACM. doi: 10.1145/2517349.2522713.

[96] Fabian Vogelsteller and Vitalik Buterin. ERC-20: Token standard. https://eips.ethereum.org/EIPS/eip-20, November 2015. Final, Standards Track: ERC. Created 2015-11-19. Accessed: 2026-10-25.

[97] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper, 2014. URL https://ethereum.github.io/yellowpaper/paper.pdf. Accessed: 2026-10-22.

[98] Gavin Wood. Polkadot: Vision for a heterogeneous multi-chain framework. *White Paper*, 21, 2016.

[99] Yu Xia, Xiangyao Yu, William Moses, Julian Shun, and Srinivas Devadas. LiTM: a lightweight deterministic software transactional memory system. In *Proceedings of the 10th International Workshop on Programming Models and Applications for Multicores and Manycores*, pages 1–10, 2019.

[100] Jiahua Xu, Damien Ackerer, and Alevtina Dubovitskaya. A game-theoretic analysis of cross-chain atomic swaps with HTLCs. In *2021 IEEE 41st international conference on distributed computing systems (ICDCS)*, pages 584–594. IEEE,

2021.

[101] Yingjie Xue and Maurice Herlihy. Hedging against sore loser attacks in cross-chain transactions. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 155–164, 2021.

[102] Anatoly Yakovenko. Solana: A new architecture for a high performance blockchain v0.8.13. Whitepaper, 2018. URL `https://solana.com/solana-whitepaper.pdf`. Accessed: 2026-10-22.

[103] Zachary Yedidia, Geoffrey Ramseyer, and David Mazières. Deterministic client: Enforcing determinism on untrusted machine code. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 633–649, 2025.

[104] Wei Yu, Kan Luo, Yi Ding, Guang You, and Kai Hu. A parallel smart contract model. In *Proceedings of the 2018 International Conference on Machine Learning and Machine Intelligence*, pages 72–77, 2018.

[105] Sergio A. Yuhjtman. Walraswap: a solution to uniform price batch auctions, 2023. URL `https://arxiv.org/abs/2310.12255`.

[106] Ryan Zarick, Isaac Zhang, Daniel Wong, Thomas Kim, Bryan Pellegrino, Mignon Li, and Kelvin Wong. FAFO: Over 1 million TPS on a single node running EVM while still Merkleizing every block, 2025. URL `https://arxiv.org/abs/2507.10757`.

[107] Jianting Zhang, Zicong Hong, Xiaoyu Qiu, Yufeng Zhan, Song Guo, and Wuhui Chen. SkyChain: A deep reinforcement learning-empowered dynamic blockchain sharding system. In *49th International Conference on Parallel Processing-ICPP*, pages 1–11, 2020.

[108] Shijie Zhang, Ru Cheng, Xinpeng Liu, Jiang Xiao, Hai Jin, and Bo Li. Seer: Accelerating blockchain transaction execution by fine-grained branch prediction. *Proceedings of the VLDB Endowment*, 18(3):822–835, 2024.